Information Processing Letters 35 (1990) 73–78
North-Holland

29 June 1990

ON A CYCLIC STRING-TO-STRING CORRECTION PROBLEM

Maurice MAES

Philips Research Laboratories, P.O. Box 80.000, 5600 JA Eindhoven, Netherlands

Communicated by G.R. Andrews
Received 16 June 1989
Revised 22 January 1990

Keywords: Analysis of algorithms, string correction, cyclic strings, pattern matching

1. Introduction

In 1974 Wagner and Fischer [1] defined a distance measure on strings, based on minimum-cost sequences of edit operations needed to change one string into another, and they presented an algorithm that computes this distance between two strings A and B in $O(nm)$ time where n and m are the lengths of A and B. As a general reference on string-to-string correction problems and applications, we mention [2]. String-to-string correction techniques can be applied to pattern recognition; see [3–6] for instance. By encoding polygonal objects as strings, one can use distance functions on these strings as a similarity measure for the objects they represent. In this application we often in fact consider two strings $a_1a_2 \dots a_n$ and $a_ka_{k+1} \dots a_na_1 \dots a_{k-1}$ to be equivalent in the sense that they represent the same polygonal object. This leads to the notion of a *cyclic* string, and in this paper we present an $O(nm \log m)$ algorithm to solve the string-to-string correction problem for cyclic strings.

2. Preliminaries: linear string-to-string correction

The definitions and results of this section are all given in [1], therefore proofs will be omitted.

Let Σ be a set of symbols and let Σ^* be the set of all finite strings over Σ. Let Λ denote the *null string*. For a string $A = a_1a_2 \dots a_n \in \Sigma^*$, and for all $i, j \in \{1, 2, \dots, n\}$, let $A\langle i, j\rangle$ denote the string $a_ia_{i+1} \dots a_j$, where, by convention $A\langle i, j\rangle = \Lambda$ if $i > j$.

An *edit operation* s is an ordered pair $(a, b) \neq (\Lambda, \Lambda)$ of strings, each of length less than or equal to 1, denoted by $a \to b$. An edit operation $a \to b$ will be called an *insert* operation if $a = \Lambda$, a *delete* operation if $b = \Lambda$, and a *change* operation otherwise.

We say that a string B results from a string A by the edit operation $s = (a \to b)$, denoted by $A \to B$ via s, if there are strings C and D such that $A = CaD$ and $B = CbD$. An *edit sequence* $S := s_1s_2 \dots s_k$ is a sequence of edit operations. We say that S *takes* A *to* B if there are strings $A_0, A_1, \dots, A_k$ such that $A_0 = A$, $A_k = B$ and $A_{i-1} \to A_i$ via s_i for all $i \in \{1, 2, \dots, k\}$.

Now let γ be a cost function that assigns a nonnegative real number $\gamma(s)$ to each edit operation s. For an edit sequence S as above, we define the cost $\gamma(S)$ by $\gamma(S) := \sum_{i=1}^{k} \gamma(s_i)$. The *edit distance* $\delta(A, B)$ from string A to string B is now defined by

$$\delta(A, B) := \min\{\gamma(S) \mid S \text{ is an edit sequence taking } A \text{ to } B\}.$$

Wagner and Fischer [1] have shown that (under the assumption that $\gamma(a \to b) = \delta(a, b)$) we can compute the edit distance between two strings $A = a_1a_2 \dots a_n$ and $B = b_1b_2 \dots b_m$, by considering

only those edit sequences that are associated with a *trace*.

Definition 2.1. Let A and B be two strings of length n and m respectively. Then a *trace* T *from* A *to* B is a set of ordered pairs of integers (i, j) with $i \in \{1, 2, \ldots, n\}$ and $j \in \{1, 2, \ldots, m\}$, such that for any two distinct pairs (i_1, j_1) and (i_2, j_2) in T we have

(a) $i_1 \neq i_2$ and $j_1 \neq j_2$,

(b) $i_1 < i_2$ if and only if $j_1 < j_2$.

A trace T can be visualized as a set of noncrossing lines between symbols of A and symbols of B: a pair $(i, j) \in T$ corresponds to a line between a_i and b_j. In an obvious way, we can associate an edit sequence S_T taking A to B with a trace T. This edit sequence is obtained by taking all change operations $a_i \to b_j$ for all pairs $(i, j) \in T$, and by deleting resp. inserting all remaining symbols in A resp. B. (See Fig. 1 for example.) In [1], traces are used to show that a minimum cost edit sequence taking A to B can be computed in the following way.

For all $i \in \{0, 1, \ldots, n\}$, $j \in \{0, 1, \ldots, m\}$, let the number $\Delta(i, j)$ be defined by

$$\Delta(i, j) := \delta(A\langle 1, i\rangle, B\langle 1, j\rangle).$$

Then it can easily be seen that, for all $i \in \{1, 2, \ldots, n\}$, $j \in \{1, 2, \ldots, m\}$,

$$\Delta(i, j) = \min \begin{cases} \Delta(i, j-1) + \gamma(\Lambda \to b_j), \\ \Delta(i-1, j) + \gamma(a_i \to \Lambda), \\ \Delta(i-1, j-1) + \gamma(a_i \to b_j). \end{cases} \quad (1)$$

Equation (1) now allows us to compute each value of $\Delta(i, j)$ in constant time (once the values $\Delta(i, j-1)$, $\Delta(i-1, j)$ and $\Delta(i-1, j-1)$ are known). Since $\Delta(n, m) = \delta(A, B)$, we see that we can compute $\delta(A, B)$ in $(n+1)(m+1)$ steps. If we are interested in a minimum-cost sequence S that takes A to B, then, after the $\Delta(i, j)$-values have been computed, we can search backwards from $\Delta(n, m)$ to $\Delta(0, 0)$, determining S in at most $n + m + 2$ steps.

Determining $\delta(A, B)$ in this way can in fact be seen as determining a minimum weighted path in a weighted directed graph [1]. Consider the weighted directed graph $G = (V, E)$, which we will call *the edit graph associated with* A *and* B, with vertices $v(i, j)$, for all $i \in \{0, 1, \ldots, n\}$, $j \in \{0, 1, \ldots, m\}$, and with the following arcs:

(i) $(v(i, j), v(i, j+1))$ with weight $w_{0,j+1} := \gamma(\Lambda \to b_{j+1})$ for all $i \in \{0, 1, \ldots, n\}$, $j \in \{0, 1, \ldots, m-1\}$,

(ii) $(v(i, j), v(i+1, j))$ with weight $w_{i+1,0} := \gamma(a_{i+1} \to \Lambda)$ for all $i \in \{0, 1, \ldots, n-1\}$, $j \in \{0, 1, \ldots, m\}$,

(iii) $(v(i, j), v(i+1, j+1))$ with weight $w_{i+1,j+1} := \gamma(a_{i+1} \to b_{j+1})$ for all $i \in \{0, 1, \ldots, n-1\}$, $j \in \{0, 1, \ldots, m-1\}$.

Figure 1 shows an example of such a graph G. Note that the arcs of type (i), (ii) and (iii) correspond to insertions, deletions and changes respectively. The problem of finding a minimum-cost edit sequence taking A to B is now reduced to finding a minimum weighted path in G from $v(0, 0)$ to $v(n, m)$.

3. Cyclic strings

A *cyclic shift* σ is a mapping $\sigma: \Sigma^* \to \Sigma^*$, defined by

$$\sigma(a_1 a_2 \ldots a_n) = a_2 a_3 \ldots a_n a_1.$$

Let σ^k denote the composition of k cyclic shifts for all $k \in \mathbf{N}$ and let σ^0 denote the identity. We will call two strings A and $\tilde{A}$ in Σ^* *equivalent* if $A = \sigma^k(\tilde{A})$ for some $k \in \mathbf{N}$. Clearly this defines an equivalence relation on Σ^*. The equivalence class of a string A will be denoted by $[A]$, and it will be called a *cyclic string*.

Given two finite strings A and B as above, we are now ready to define the edit distance $\delta([A], [B])$ between the cyclic strings they represent:

$$\delta([A], [B]) := \min\{\delta(\sigma^k(A), \sigma^l(B)) \mid k, l \in \mathbf{N}\}.$$

Note that this edit distance is well-defined. Similarly, we define the edit distance $\delta(A, [B])$ between a linear and a cyclic string by

$$\delta(A, [B]) := \min\{\delta(A, \sigma^l(B)) \mid l \in \mathbf{N}\}.$$

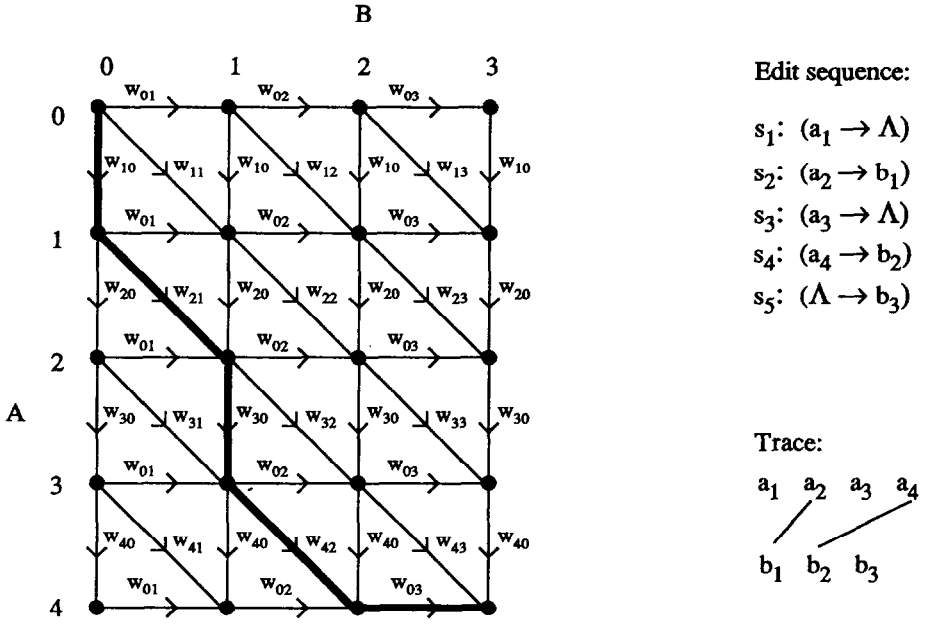

Fig. 1. An edit graph, a path and its corresponding edit sequence and trace.

The following lemma shows that in order to compute the edit distance between two cyclic strings, one can simply choose a representative of one cyclic string, and compute the edit distance between this linear string and the other cyclic string.

Lemma 3.1. *For each representative A of a cyclic string $[A]$ and for each cyclic string $[B]$ we have $\delta([A], [B]) = \delta(A, [B])$.*

Proof. The proof of this lemma is straightforward, so we will leave the details to the reader. First note that it suffices to show that $\delta(A, [B]) \geqslant \delta(\sigma(A), [B])$. Then consider a trace T_1 that represents a minimum-cost edit sequence taking A to $\sigma^{l_1}(B)$, for some l_1, and show that there is a trace T_2 taking $\sigma(A)$ to $\sigma^{l_2}(B)$, for some l_2, such that T_2 represents exactly the same set of edit operations as T_1. □

Now the problem that we wish to solve is the following.

Problem 3.2. Given two strings A and B, determine $\delta([A], [B])$ and an edit sequence S between two representatives of $[A]$ and $[B]$ realizing this minimum cost.

From Lemma 3.1 it follows that it suffices to solve the following problem.

Problem 3.3. Given two strings A and B, determine $\delta(A, [B])$ and an edit sequence S between A and a representative of $[B]$ realizing this minimum cost.

One way to solve this problem is to determine for each $l \in \{1, 2, \ldots, m\}$ a minimum-cost sequence between A and $\sigma^l(B)$ by the Wagner and Fischer algorithm. This would lead to an $O(nm^2)$ algorithm. In the next section we present an $O(nm \log m)$ algorithm to solve this problem.

4. An $O(nm \log m)$ algorithm

Suppose we have two strings A and B of lengths n and m respectively. By symmetry, we may without loss of generality assume that $m \leqslant n$. In order to solve Problem 3.3 we proceed as follows.

First note that for all $l \in \{1, 2, \ldots, m\}$ the edit graph $G_l = (V_l, E_l)$ associated with A and $\sigma^l(B)$ is in fact a "cyclic shift" of the graph $G = (V, E)$ associated with A and B: the "columns" of the graph are "cyclically shifted".

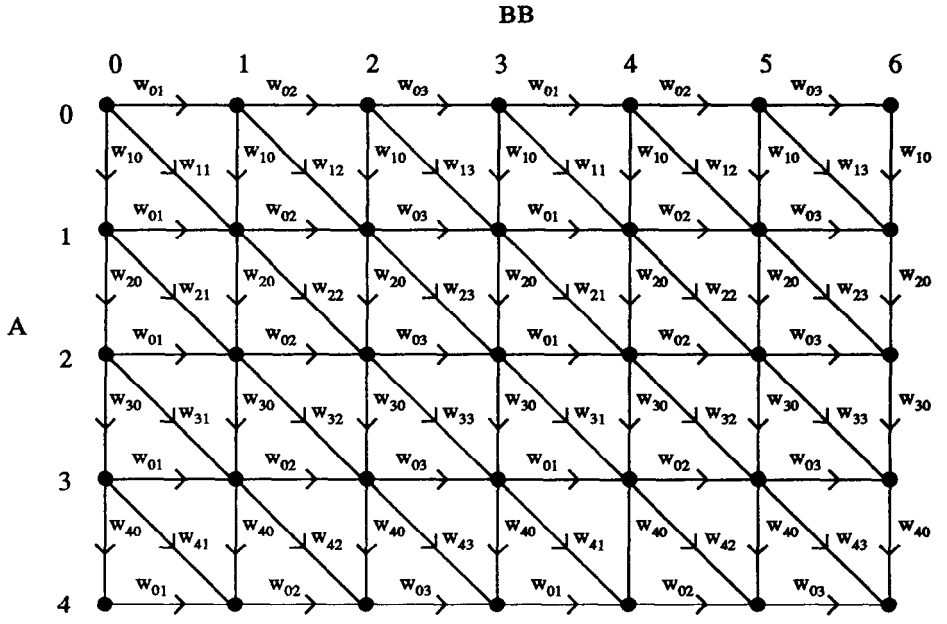

Fig. 2. The graph H associated with A and BB in the case where $|A| = 4$ and $|B| = 3$.

Next consider the edit graph H associated with the strings A and $BB = b_1b_2 \ldots b_mb_1b_2 \ldots b_m$, the concatenation of B with itself. This graph has $(n+1)(2m+1)$ vertices, and it in fact "includes" all graphs G_l; see Fig. 2.

Property 4.1. *For each $l \in \{0, 1, \ldots, m\}$, if we remove from H all arcs and vertices that do not lie on any path from vertex $v(0, l)$ to vertex $v(n, m+l)$, then the remaining weighted graph is equal to the graph $G_l(V_l, E_l)$.*

We see that, in order to find a minimum-cost edit sequence from A to $\sigma^l(B)$, we can determine a minimum weighted path within H from $v(0, l)$ to $v(n, m+l)$. Before we can make a second observation, we need a few more definitions.

Definition 4.2. For all $l \in \{0, 1, \ldots, m\}$, $P(l)$ will denote a path from $v(0, l)$ to $v(n, m+l)$. Let j, k, and l be three integers such that $0 \leqslant j < k < l \leqslant m$. We say that two paths $P(k)$ and $P(l)$ are *noncrossing* if for all $x \in \{0, 1, \ldots, n\}$, $y \in \{0, 1, \ldots, 2m\}$ the following two conditions are satisfied:

(i) if $v(x, y) \in P(k)$, then there is a $z \in \mathbf{N}$ with $z \geqslant y$ such that $v(x, z) \in P(l)$,

(ii) if $v(x, y) \in P(l)$, then there is a $z \in \mathbf{N}$ with $z \leqslant y$ such that $v(x, z) \in P(k)$.

Furthermore, given three paths $P(j)$, $P(k)$, and $P(l)$, we say that $P(k)$ is *between* $P(j)$ and $P(l)$ if $P(j)$ and $P(k)$, as well as $P(k)$ and $P(l)$ are noncrossing.

Note that if two paths are noncrossing, then they may partly coincide. Now the second important observation is the following.

Lemma 4.3. *Let j, k, and l be three integers such that $0 \leqslant j < k < l \leqslant m$, and let $P(j)$ and $P(l)$ be two noncrossing minimum weighted paths in H. Then there is a minimum weighted path $P(k)$ that lies between $P(j)$ and $P(l)$.*

Proof. This is not difficult and left to the reader: use the fact that if two paths are not noncrossing, then they have at least two vertices in common, and if one of the paths is of minimum weight, then the subpath connecting the two vertices is also of minimum weight. □

The computation of the path then involves the calculation of the Δ-values, (using Eq. (1)), for

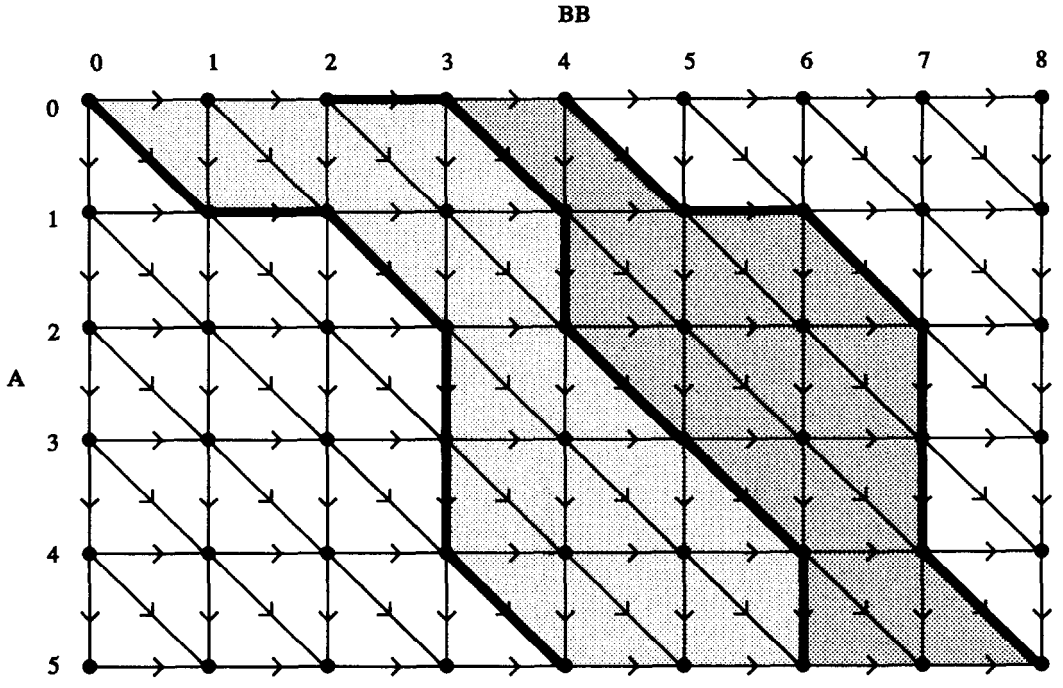

Fig. 3. An illustration of the algorithm, in the case where $|A| = 5$ and $|B| = 4$.

those vertices v such that there is a path $P(k)$ through v that is between $P(j)$ and $P(l)$.

From now on we will assume that $m = 2^q$ for some $q \in \mathbf{N}$. This leads to no loss of generality; it is done for convenience only. The following algorithm computes all minimum weighted paths $P(i)$ all $i \in \{0, 1, \ldots, m\}$, and therefore solves Problem 3.3.

The algorithm

Step 1: Determine a minimum weighted path $P(0)$ and the corresponding (equal) minimum weighted path $P(m)$;
Step 2:
for j **from** 1 **to** q
do for k **from** 1 **to** 2^{j-1}
  do compute a minimum weighted path $P((2k-1)m/2^j)$ between $P((2k-2)m/2^j)$ and $P(2km/2^j)$;
  od
od

The correctness of this algorithm is easily seen: it depends on Lemma 4.3. An example is given in Fig. 3: After $P(0)$ has been computed, the computation of $P(2)$ involves only vertices that lie within the grey area or on the paths $P(0)$ or $P(4)$. Next, $P(1)$ can be computed between $P(0)$ and $P(2)$, so only vertices on these paths or within the light grey area need to be considered. Similarly $P(3)$ can be computed between $P(2)$ and $P(4)$.

Time and space analysis

There are two types of elementary steps in this algorithm that are important as far as the time complexity of the algorithm is concerned.

Type (a): *the calculations of the $\Delta(i, j)$-values of the vertices $v(i, j)$ for a given path.* In order to give a bound for the number of steps of type (a), we first note that Step 1 involves $(n+1)(m+1)$ steps of this type. Next, and this is crucial, note that *within one j-loop of the algorithm, each vertex of H can occur in at most two paths that are calculated within that loop.* (A vertex can occur in two paths only if it lies on a previously calculated path.) So at most two calculations of type (a) are needed for each vertex during one j-loop. Since the total number of vertices equals $(n+1)(2m+$

1), this leads to a total of less than $2(n+1)(2m+1)$ steps of type (a) for each of the $q = \log m$ j-loops. We see that $O(nm \log m)$ steps of type (a) are needed.

Type (b): *The comparisons involved in the (backward) calculation of a minimum weighted path*. Since any path has length at most $n+m+2$, determining all m different minimum weighted paths takes at most $m(n+m+2)$ steps of type (b).

Combining these results gives the following theorem.

Theorem 4.4. *The algorithm given above solves Problem* 3.3 *in* $O(nm \log m)$ *time.*

Finally, it should be noted that the above algorithm consumes $O(nm)$ space as described. However, one can reduce this to $O((n+m)\log m)$ space, by using the divide-and-conquer approach of Hirschberg [7] for finding each single path, and by calculating the paths in the proper order, i.e. such that at most $O(\log m)$ previously calculated paths have to be stored.

Acknowledgment

The author wishes to thank Ernst van der Plas and Henk Hollmann for discussions on this subject, as well as the anonymous referee for his valuable suggestions.

References

[1] R.A. Wagner and M.J. Fischer, The string-to-string correction problem, *J. Assoc. Comput. Mach.* **21** (1) (1974) 168–173.

[2] D. Sankoff and J.B. Kruskal, eds., *Time Warps, String Edits and Macromolecules: The Theory and Practice of Sequence Comparison* (Addison-Wesley, Reading, MA, 1983).

[3] T.I. Fan, Optimal matching of deformed patterns with positional influence, *Inform. Sci.* **41** (1987) 259–280.

[4] S.Y. Lu and K.S. Fu, Stochastic error-correcting syntax analysis for recognition of noisy patterns, *IEEE Trans. Comput.* **26** (12) (1977) 1268–1276.

[5] H.-C. Liu, M.D. Srinath, Classification of partial shapes using string-to-string matching, *Intelligent Robots and Computer Vision*, SPIE Proceedings Vol. 1002 (1989) 92–98.

[6] W.H. Tsai and S.S. Yu, Attributed string matching with merging for shape recognition, *IEEE Trans. Patt. Anal. Mach. Intell.* **7** (4) (1985) 453–462.

[7] D.S. Hirschberg, A linear space algorithm for computing maximal common subsequences, *Comm. ACM* **18** (1975) 341–343.